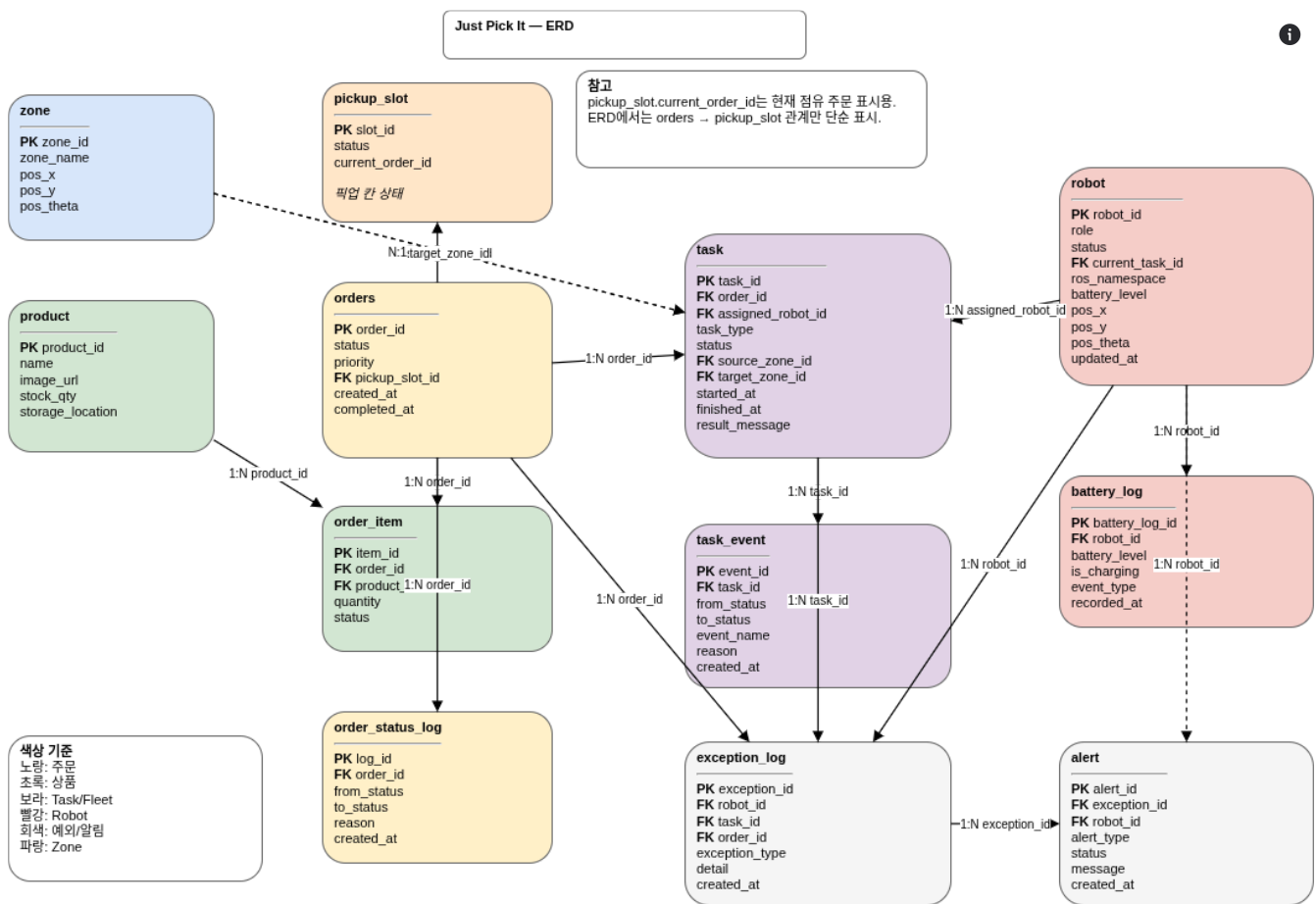


Data Structure

1. 전체 데이터 흐름

```
1 고객 주문 생성
2 → orders 생성
3 → order_item 생성
4 → task 생성
5 → Fleet Manager가 task를 로봇에 할당
6 → robot 상태 갱신
7 → task_event 기록
8 → order_status_log 기록
9 → 검수 완료 후 pickup_slot 배정
10 → 고객 수령 완료
11 → 예외 발생 시 exception_log + alert 기록
```

2. ERD 개요



3. 테이블 역할 요약

테이블	역할	사용 위치
zone	로봇이 이동할 주요 위치 정보	Fleet Manager, AMR 목적지

product	상품 정보와 재고	고객 상품 목록, 관리자 재고 관리
pickup_slot	픽업 칸 상태 관리	픽업존, 고객 수령
orders	주문 1건의 대표 상태	고객 주문 상태, 관리자 주문 현황
order_item	주문 안에 포함된 상품 목록	Cobot 선별 / 검수 목록
robot	로봇 4대의 현재 상태	관리자 관제, Fleet Manager
task	로봇에게 줄 작업 단위	작업 큐, 자동 작업 배정
task_event	작업 진행 이력	관리자 작업 타임라인
order_status_log	주문 상태 변경 이력	고객 진행 상태, 디버깅
battery_log	AMR 배터리 이력	배터리 관리, 충전 판단
exception_log	예외 상황 기록	장애 분석, 복구 처리
alert	관리자 알림	관리자 대시보드

4. ENUM 타입 정의

```

1 CREATE TYPE order_status AS ENUM (
2     'ORDER_RECEIVED', -- 주문 접수 (고객 주문 상태 UI, 관리자 주문 현황)
3     'ORDER_WAIT',    -- 주문 대기 (Fleet Manager 작업 대기)
4     'SORTING',       -- 상품 선별 중 (고객 진행 상태 UI, 관리자 주문 현
    황)
5     'DELIVERING',    -- 상품 운반 중 (관리자 관제 화면, 고객 진행 상태
    UI)
6     'INSPECTING',    -- 상품 검수 중 (관리자 주문 현황)
7     'PICKUP_READY',  -- 픽업 준비 완료 (고객 픽업 안내 UI)
8     'COMPLETED',    -- 수령 완료 (주문 이력)
9     'ERROR'         -- 예외 발생 (관리자 알림, 고객 오류 메시지)
10 );
11
12 CREATE TYPE order_item_status AS ENUM (
13     'WAITING',        -- 작업 대기 중 (Cobot1 작업 대기)
14     'SORTED',        -- Cobot1이 AMR에 선별 상차 완료 (관리자 주문 상세)
15     'INSPECTED',     -- Cobot2 상품 검수 완료 (관리자 주문 상세)
16     'MISSING',       -- 검수 결과 상품 누락 (exception_log, 관리자 알림)
17     'EXCESS',        -- 검수 결과 상품 초과 (exception_log, 관리자 알림)
18     'MISSMATCH'     -- 검수 결과 불일치 (exception_log, 관리자 알림)
19 );
20
21 CREATE TYPE pickup_slot_status AS ENUM (
22     'EMPTY',         -- 비어 있음 (픽업 칸 자동 배정)
23     'RESERVED',     -- 주문에 배정됨 (Fleet Manager)
24     'OCCUPIED',     -- 상품 있음 (고객 픽업 안내 UI)

```

```

25     'BLOCKED'          -- 사용 불가 (관리자 픽업존 관리)
26 );
27
28 CREATE TYPE robot_role AS ENUM (
29     'SORTION',          -- Cobot1: 상품 선별 및 상차 (진열대 -> AMR)
30     'INSPECTION',       -- Cobot2: 상품 검수 및 하차 (AMR -> 픽업 슬롯)
31     'TRANSPORT',       -- AMR: 선별상차 구역 ↔ 검수하차 구역 운반
32     'PATROL'           -- AMR: 순찰 및 비상 상황 감지
33 );
34
35 CREATE TYPE robot_status AS ENUM (
36     'IDLE',             -- 대기 중 (관리자 관제 화면)
37     'MOVING',           -- 이동 중 (AMR 실시간 상태)
38     'SORTING',          -- 선별 작업 수행 중 (관리자 관제 화면)
39     'DELIVERING',       -- 상품 운반 중 (관리자 관제 화면)
40     'INSPECTING',       -- 검수 작업 수행 중 (관리자 관제 화면)
41     'PATROLLING',       -- 순찰 중 (관리자 관제 화면)
42     'CHARGING',         -- 충전 중 (배터리 관리 UI)
43     'RETURNING',        -- 대기구역 복귀 중 (Fleet Manager)
44     'EMERGENCY_STOP',   -- 긴급 정지 상태 (안전 시스템)
45     'ERROR',            -- 로봇 오류 상태 (관리자 알림)
46     'OFFLINE'          -- 연결 끊김 (관리자 알림)
47 );
48
49 CREATE TYPE task_type AS ENUM (
50     'SORTION',          -- 상품 선별 및 상차 (Cobot1)
51     'DELIVERY',         -- 상품 운반 (AMR_TRANSPORT)
52     'INSPECTION',       -- 상품 검수 (Cobot2)
53     'PATROL',           -- 순찰 작업 (AMR_PATROL)
54     'RETURN_HOME'       -- 대기구역 복귀 (AMR 로봇)
55     'CHARGE',           -- 충전 (AMR)
56 );
57
58 CREATE TYPE task_status AS ENUM (
59     'QUEUED',           -- 작업 큐 대기 중 (Fleet Manager)
60     'ASSIGNED',         -- 특정 로봇에 할당됨 (Fleet Manager)
61     'RUNNING',          -- 현재 수행 중 (관리자 작업 타임라인)
62     'PAUSED',           -- 긴급 정지 등으로 일시 중단 (안전 시스템)
63     'SUCCESS',          -- 작업 성공 완료 (작업 로그)
64     'FAILED',           -- 작업 실패 (exception_log)
65     'CANCELLED'         -- 작업 취소 (작업 로그)
66 );
67
68 CREATE TYPE exception_type AS ENUM (
69     'OBSTACLE_DETECTED', -- 장애물 감지 (관리자 알림)
70     'LOW_BATTERY',       -- 배터리 부족 (배터리 관리)
71     'NAVIGATION_FAILED', -- 경로 탐색 또는 주행 실패 (exception_log)
72     'HARDWARE_ERROR',    -- 하드웨어 오류 (관리자 알림)
73     'TIMEOUT',           -- 작업 시간 초과 (Fleet Manager)
74     'SORTION_FAIL',      -- 상품 선별 실패 (관리자 알림)
75     'INSPECTION_FAIL',   -- 검수 실패 (관리자 알림)
76     'HUMAN_DETECTED',    -- 사람 감지 (긴급 정지 시스템)
77     'SYSTEM_ERROR'       -- 시스템 오류 (관리자 알림)
78 );
79
80 CREATE TYPE alert_type AS ENUM (
81     'ROBOT_ERROR',       -- 로봇 오류 발생 (관리자 대시보드)
82     'LOW_BATTERY',       -- 배터리 부족 (관리자 대시보드)
83     'TASK_FAILED',       -- 작업 실패 (관리자 대시보드)
84     'OBSTACLE_DETECTED', -- 장애물 감지 (관리자 대시보드)
85     'SORTION_FAIL',      -- 상품 선별 실패 (관리자 대시보드)
86     'INSPECTION_FAILED', -- 검수 실패 (관리자 대시보드)
87     'SYSTEM_WARNING'     -- 시스템 경고 (관리자 대시보드)
88 );
89
90 CREATE TYPE alert_status AS ENUM (
91     'UNREAD',            -- 아직 확인 안 함 (관리자 알림 목록)

```

```

92     'READ',          -- 관리자 확인 완료 (관리자 알림 목록)
93     'RESOLVED'       -- 처리 완료 (관리자 알림 목록)
94 );

```

5. 테이블 정의

5-1. zone

zone 은 로봇이 이동하거나 작업하는 주요 위치를 저장한다.

AMR은 이 테이블의 좌표를 기준으로 선별상차 구역, 검수하차 구역, 흙, 충전 구역으로 이동한다.

```

1 CREATE TABLE zone (
2     zone_id      SERIAL PRIMARY KEY,
3     zone_name    VARCHAR(50) NOT NULL,
4     pos_x        FLOAT NOT NULL,
5     pos_y        FLOAT NOT NULL,
6     pos_z        FLOAT NOT NULL,
7     pos_theta    FLOAT
8 );
9
10 INSERT INTO zone (zone_name, pos_x, pos_y, pos_theta) VALUES
11 ('LOADING_ZONE', 1.0, 2.0, 1.0, 0.0),
12 ('UNLOADING_ZONE', 3.0, 4.0, 1.0, 0.0),
13 ('HOME', 0.0, 0.0, 1.0, 0.0),
14 ('CHARGING_ZONE', 0.5, 0.5, 1.0, 0.0),
15 ('Product_ZONE', 1.0, 2.0, 1.0, 0.0); -- 상품 별 위치 값 지정

```

5-2. product

product 는 상품 정보와 재고를 저장한다.

고객 UI에서는 상품명, 이미지, 재고를 보여주고, 관리자 UI에서는 재고를 수정한다.

```

1 CREATE TABLE product (
2     product_id   SERIAL PRIMARY KEY,
3     name         VARCHAR(100) NOT NULL,
4     image_url    TEXT,
5     stock_qty    INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0),
6     storage_location VARCHAR(50) NOT NULL
7 );

```

5-3. pickup_slot

pickup_slot 은 고객이 상품을 가져갈 픽업 칸을 관리한다.

픽업 칸은 주문과 연결될 수 있으며, 수령 완료 후 다시 빈 칸으로 초기화된다.

```

1 CREATE TABLE pickup_slot (
2     slot_id      SERIAL PRIMARY KEY,
3     slot_name    VARCHAR(50), -- 추가: 화면 표시용 픽업 칸 이름
4     status       pickup_slot_status NOT NULL DEFAULT 'EMPTY',
5     current_order_id INT
6 );
7
8 INSERT INTO pickup_slot (slot_name, status) VALUES
9 ('Pickup_slot_1', 'EMPTY'),
10 ('Pickup_slot_2', 'EMPTY');

```

5-4. orders

`orders` 는 주문 1건의 대표 정보를 저장한다.

고객 입장에서는 “내 주문이 지금 어디까지 진행됐는지”를 확인하는 기준이고, 관리자 입장에서는 전체 주문 현황을 보는 기준이다.

```
1 CREATE TABLE orders (
2     order_id      SERIAL PRIMARY KEY,
3     order_no      VARCHAR(30) UNIQUE, -- 추가: 고객/관리자 표시용 주문 번
4     호
5     status        order_status NOT NULL DEFAULT 'ORDER_RECEIVED',
6     priority      INT NOT NULL DEFAULT 1,
7     pickup_slot_id INT REFERENCES pickup_slot(slot_id)
8 );
9
10 ALTER TABLE pickup_slot
11     ADD CONSTRAINT fk_pickup_slot_order
12     FOREIGN KEY (current_order_id) REFERENCES orders(order_id) ON
13     DELETE SET NULL;
```

5-5. order_item

`order_item` 은 주문 안에 들어 있는 상품 목록을 저장한다.

예를 들어 고객이 콜라 2개, 과자 1개를 주문하면 `orders` 는 1개 생성되고, `order_item` 은 2개 생성된다.

```
1 CREATE TABLE order_item (
2     item_id      SERIAL PRIMARY KEY,
3     order_id      INT NOT NULL REFERENCES orders(order_id) ON DELETE
4     CASCADE,
5     product_id    INT NOT NULL REFERENCES product(product_id),
6     quantity      INT NOT NULL CHECK (quantity > 0),
7     status        order_item_status NOT NULL DEFAULT 'WAITING'
8 );
```

5-6. robot

`robot` 은 로봇 4대의 현재 상태를 저장한다.

관리자 관제 화면과 Fleet Manager가 가장 자주 참조하는 테이블이다.

```
1 CREATE TABLE robot (
2     robot_id      VARCHAR(30) PRIMARY KEY,
3     role          robot_role NOT NULL,
4     status        robot_status NOT NULL DEFAULT 'IDLE',
5     current_task_id INT REFERENCES task(task_id),
6     ros_namespace VARCHAR(50),
7     battery_level  INT CHECK (battery_level BETWEEN 0 AND 100),
8     pos_x         FLOAT,
9     pos_y         FLOAT,
10    pos_theta      FLOAT
11 );
12
13 INSERT INTO robot (robot_id, role, status, ros_namespace,
14    battery_level) VALUES
15    ('COBOT1', 'SORTION', 'IDLE', '/cobot1', NULL),
16    ('COBOT2', 'INSPECTION', 'IDLE', '/cobot2', NULL),
17    ('AMR_TRANSPORT', 'TRANSPORT', 'IDLE', '/amr1', 100),
18    ('AMR_PATROL', 'PATROL', 'IDLE', '/amr2', 100);
```

5-7. task

task 는 로봇이 실제로 수행할 작업 단위다.

orders 가 고객 관점의 주문이라면, task 는 로봇 관점의 실행 명령이다.

```
1 CREATE TABLE task (  
2     task_id          SERIAL PRIMARY KEY,  
3     order_id         INT REFERENCES orders(order_id) ON DELETE  
4     CASCADE,  
5     assigned_robot_id VARCHAR(30) REFERENCES robot(robot_id),  
6     task_type        task_type NOT NULL,  
7     status           task_status NOT NULL DEFAULT 'QUEUED',  
8     source_zone_id   INT REFERENCES zone(zone_id),  
9     target_zone_id   INT REFERENCES zone(zone_id),  
10    result_message    TEXT  
11 );  
12 ALTER TABLE robot  
13     ADD CONSTRAINT fk_robot_current_task  
14     FOREIGN KEY (current_task_id) REFERENCES task(task_id);
```

5-8. task_event

task_event 는 작업 상태 변화와 주요 이벤트를 기록한다.

예를 들어 선별상차 작업 하나 안에서도 작업 할당, 작업 시작, 상품 상차 완료, 작업 성공 같은 이벤트가 생길 수 있다.

```
1 CREATE TABLE task_event (  
2     event_id         SERIAL PRIMARY KEY,  
3     task_id          INT NOT NULL REFERENCES task(task_id) ON DELETE  
4     CASCADE,  
5     robot_id         VARCHAR(30) REFERENCES robot(robot_id), -- 추가: 이벤트  
6     from_status       task_status,  
7     to_status        task_status NOT NULL,  
8     event_name        VARCHAR(50),  
9     reason           TEXT,  
10    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()  
11 );
```

5-9. order_status_log

order_status_log 는 주문 상태 변경 이력을 저장한다.

orders.status 는 현재 상태만 저장하고, order_status_log 는 과거 상태 변화 기록을 저장한다.

```
1 CREATE TABLE order_status_log (  
2     log_id          SERIAL PRIMARY KEY,  
3     order_id        INT NOT NULL REFERENCES orders(order_id) ON DELETE  
4     CASCADE,  
5     from_status     order_status,  
6     to_status       order_status NOT NULL,  
7     reason          TEXT,  
8     created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()  
9 );
```

5-10. battery_log

battery_log 는 AMR 배터리 이력을 저장한다.

robot.battery_level 은 현재 배터리이고, battery_log 는 시간별 기록이다.

```
1 CREATE TABLE battery_log (  
2     battery_log_id SERIAL PRIMARY KEY,  
3     robot_id VARCHAR(30) NOT NULL REFERENCES robot(robot_id),  
4     battery_level INT NOT NULL CHECK (battery_level BETWEEN 0 AND  
5     100),  
6     is_charging BOOLEAN NOT NULL DEFAULT FALSE,  
7     event_type VARCHAR(30),  
8     recorded_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
9 );
```

5-11. exception_log

exception_log 는 로봇이나 시스템에서 발생한 문제를 기록한다.

예외 발생 시 관리자 알림을 만들고, 필요하면 주문이나 작업을 중단한다.

```
1 CREATE TABLE exception_log (  
2     exception_id SERIAL PRIMARY KEY,  
3     robot_id VARCHAR(30) REFERENCES robot(robot_id),  
4     task_id INT REFERENCES task(task_id),  
5     order_id INT REFERENCES orders(order_id),  
6     exception_type exception_type NOT NULL,  
7     detail TEXT,  
8     created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
9 );
```

5-12. alert

alert 는 관리자 화면에 표시할 알림을 저장한다.

exception_log 가 실제 문제 기록이라면, alert 는 관리자가 보는 알림이다.

```
1 CREATE TABLE alert (  
2     alert_id SERIAL PRIMARY KEY,  
3     exception_id INT REFERENCES exception_log(exception_id),  
4     robot_id VARCHAR(30) REFERENCES robot(robot_id),  
5     alert_type alert_type NOT NULL,  
6     status alert_status NOT NULL DEFAULT 'UNREAD',  
7     message TEXT  
8 );
```

6. 인덱스 설계

```
1 CREATE INDEX idx_orders_status  
2 ON orders(status);  
3  
4 CREATE INDEX idx_order_item_order_id  
5 ON order_item(order_id);  
6  
7 CREATE INDEX idx_order_item_product_id  
8 ON order_item(product_id);  
9  
10 CREATE INDEX idx_robot_status  
11 ON robot(status);  
12  
13 CREATE INDEX idx_robot_current_task_id
```

```
14 ON robot(current_task_id);
15
16 CREATE INDEX idx_task_order_id
17 ON task(order_id);
18
19 CREATE INDEX idx_task_assigned_robot_id
20 ON task(assigned_robot_id);
21
22 CREATE INDEX idx_task_status
23 ON task(status);
24
25 CREATE INDEX idx_task_type
26 ON task(task_type);
27
28 CREATE INDEX idx_task_event_task_id
29 ON task_event(task_id);
30
31 CREATE INDEX idx_order_status_log_order_id
32 ON order_status_log(order_id);
33
34 CREATE INDEX idx_battery_log_robot_id
35 ON battery_log(robot_id);
36
37 CREATE INDEX idx_exception_log_robot_id
38 ON exception_log(robot_id);
39
40 CREATE INDEX idx_exception_log_task_id
41 ON exception_log(task_id);
42
43 CREATE INDEX idx_exception_log_order_id
44 ON exception_log(order_id);
45
46 CREATE INDEX idx_alert_status
47 ON alert(status);
48
49 CREATE INDEX idx_alert_robot_id
50 ON alert(robot_id);
```